

System and Network Security

Dr. Ashok Kumar Das

IEEE Senior Member
Professor

Center for Security, Theory and Algorithmic Research
International Institute of Information Technology, Hyderabad

E-mail: *ashok.das@iiit.ac.in*

URL: <https://www.iiit.ac.in/faculty/ashok-kumar-das/>
<https://sites.google.com/view/iitkgpakdas/>

Heap Overflow Attacks

- In languages such as C (and C++), a programmer can request a block of memory using the system call
`void * malloc (size_t size)`.
- When not needed any more, the program surrenders the block using the system call
`void free (void * pointer)`.
- The allocated blocks are in a region of memory called the *heap*.
- Initially, they are all contiguous but that would change as allocated blocks are freed.

- The **Heap Manager** keeps the track of which blocks are in use and which are free.
- Blocks are of different sizes (each is a multiple of 4 bytes).
- When an allocated block becomes free, its two adjacent blocks are checked to see if they are free. If so, the newly free block is merged with its free neighbor(s) to create a larger free block.
- Each block contains management information. The first two words of a block store its size (S) and the size of the previous block (P) (each word is 4 bytes).
- In many heap implementations, free blocks are kept track of by means of a **doubly-linked list**.
- A forward pointer (F) and a backward pointer (B) are used to implement the list.
- F points to the next free block in the list.
- B points to the previous free block in the list.

Heap Overflow Attacks

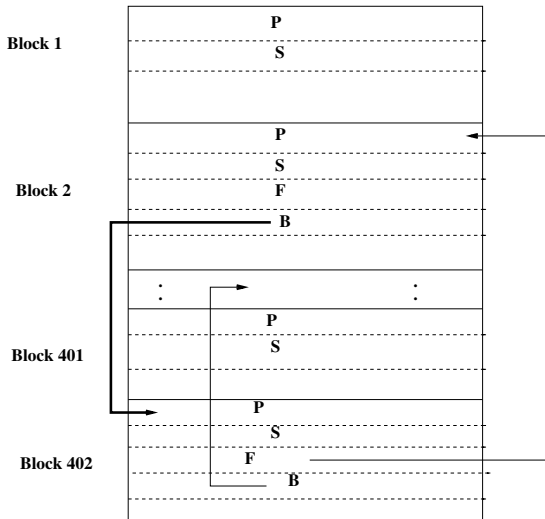


Figure: Doubly-linked list of free blocks

- BOFs on the heap occur for exactly the same reason as stack BOFs- languages such as C do not perform array bound checking.
- A program can read or write into memory locations beyond the boundary of the block allocated to it.
- This is a serious vulnerability as an attacker's program may be able to read/write sensitive information such as passwords, file names, etc. stored on other blocks not allocated to it.

An example

- Assume that Block 1 is allocated to the attacker's program and Block 2 is free.
- Since the attacker knows the size of Block 1, it is easy for him to compute the addresses of the locations containing the management information in Block 2.
- Let the attacker wish to overwrite location x with value y .
- All he needs to overwrite the F of Block 2 with $x - 12$
[$= x - 3 \times 4 \text{ bytes} = x - 12$]
[$= x - (B + F + S)$]
and B with y .

Heap Overflow Attacks

An example (Continued...)

- Consider the scenario when Block 1 is freed.
- Note that Block 1 was allocated to the attacker's program.
- Since Block 2 is already freed, Blocks 1 and 2 will be merged together.
- The merged block will be placed at the appropriate position in the linked list (free blocks are placed in the list in increasing order of size).
- Since merged block's size is large, Block 776 must be now made the successor of Block 402.
- The Heap Manager needs to overwrite *B* of Block 776 to point to Block 402.
- Similarly, it will overwrite *F* of Block 402 to point to Block 776.
- But *F* and *B* of Block 402 have been already corrupted by the attacker, i.e., they are respectively overwritten with the values $x - 12$ and y , respectively.

Heap Overflow Attacks

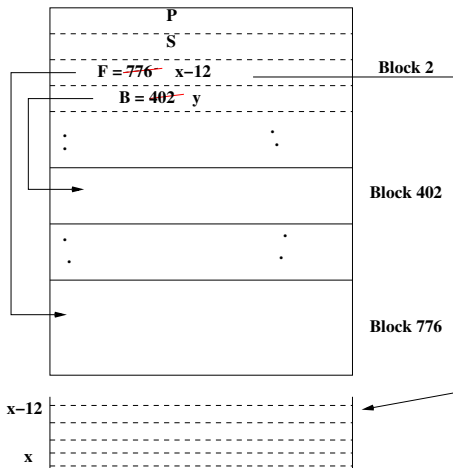


Figure: Illustrating heap corruption

Heap Overflow Defenses

- Since version 2.3.6 the GNU LibC includes protections that can detect heap overflows after the fact, for example by checking pointer consistency when calling unlink.
- Microsoft Windows operating systems implement protections against heap overflows since Windows XP SP2 uses safe unlinking and cookies. It also can migrate these threats through the use of Data Execution Prevention (DEP) and ASLR (Address space layout randomization).
- **Reference:** P. - A. Faylle and V. Glaume, "A Buffer Overflow Study: Attacks & Defenses". Available at <http://www.freeinfosociety.com/media/pdf/2708.pdf>

Format String Attacks

- C functions such as **printf()** take a format string as the first argument as in
`printf("var1 in decimal is %d and var2 in hex is %x \n", var1, var2).`
- `%d` and `%x` are referred to as format specifiers.
- Each occurrence of a `%d` or `%x`, `printf( )` expects to see a variable name in its list of arguments.
- Likewise, for each occurrence of a `%s` or `%n`, `printf( )` expects to see pointer arguments.
- For example, `printf("This course name is %s", name).`
Here `name` is a string variable, i.e., pointer to an array of characters.
- As it turns out, `%s` and `%n` are the format specifiers that can be used by an attacker to read and write arbitrary locations in memory.

Example

```
printf("# of characters printed is %n", &count);  
printf("%d", count);
```

- The first `printf()` statement causes the number of bytes printed by that statement to be output to the variable, `count`.
- The second `printf()` statement then displays the number of bytes printed.
- In the above program, output is 27.

Reading from an arbitrary memory location

```
int main (int argc, const char * argv[]) {  
    char buf[160];  
    strcpy(buf, argv[1]);  
    printf("The content of memory location 12345678 is: ");  
    printf(buf);  
    exit(0);  
}
```

- If we compile and execute the above program with the following command-line input:
./a.out "Hello %d"
we get the output as
The content of memory location 12345678 is: **Hello 134514137**

SQL Injection Attacks

The Vulnerability

- Multi-tier web applications typically have three tiers- the web, application, and database tiers.
 - ▶ The Web tier interacts directly with the application layer which, in turn, interfaces with the database tier.
For applications with limited business logic, the application tier may be fused with the web tier.
In such applications, a component in the web tier may directly communicate with the database tier.
 - ▶ The database tier contains the database and the Database Management System (DBMS).
The database is itself compromised of a number of tables (or relations).
Each relation has a number of attributes (columns) and tuples (rows).
A relation is an instance of a schema.

SQL Injection

SQL stands for **Structured Query Language**. It is designed for managing data in a relational database management system (RDBMS).

Table: An instance of the Students schema (students14)

S_ID	Name	Encrypted Password	Department	CGPA
201705514	Jairam P.	3q!ylrsj	CSE	9.89
201706617	Rahul S.	Bu73@jT	ECE	8.60
201701179	Priya D.	L189!24	CSE	9.71
⋮	⋮	⋮	⋮	⋮

Consider the following SQL query:

```
select  S_ID, CGPA
from    students14
where   S_ID = 201706617 and password = 'Bu73@jT'
```

Result: Rahul S., 8.60

The Vulnerability

- Consider a University website that permits its students to view their registration status and grade information provided they login with their correct student ID and password.
- The University web server presents a webpage containing a form to the user (HTML supports forms using a special `< form >` tag).
- The inputs entered by the user are passed to the server as form parameters when the user clicks on the form's SUBMIT button.
- Form parameters may be passed in the body of an HTTP POST request. Alternatively, they may be passed as a query string in an extended URL to the server as
`http://www.iiit.ac.in?S_ID = 201706617 & passwd = Bu73@jT`

The Vulnerability

- The server application retrieves the form parameters and uses them to build an SQL query such as

```
select  S_ID, CGPA
from    students14
where   S_ID = 201706617 and password = 'Bu73@jT'
```

- Some applications build SQL queries using string concatenation and then submit the query to the DBMS.

The Vulnerability

```
String studentID = ...; \\ student'd ID obtained from HTTP  
request packet  
String pw = ...; \\ student's password obtained from  
HTTP request packet  
Connection con = ...; \\ Connection to the DB obtained  
Statement stmt = con.createStatement();  
String query = "select CGPA  
from students17  
where S_ID = " + studentID + "and pass-  
word = " + pw + """;  
ResultSet rs = stmt.executeQuery(query);
```

Attack 1: Retrieving specific student's information [Student ID is known, but his/her password is unknown to an attacker]

IIIT Hyderabad

To obtain your most recent CGPA enter your studentID
and password below

Student ID:

Password:

The query built from the inputs supplied by the attacker is

```
select  S_ID, CGPA
from    students17
where   S_ID = 123
        and password = 'abc' or 'x' = 'x'
```

Attack 2: Retrieving specific student's information or entire table (relation) information [Student ID may or many not be known, and his/her password is certainly unknown to an attacker]

IIIT Hyderabad

To obtain your most recent CGPA enter your studentID
and password below

Student ID:

Password:

The query built from the inputs supplied by the attacker is

```
select  S_ID, CGPA
from    students17
where   S_ID = 123 or 1 = 1 - -
        and password = 'abc'
```

Attack 3: Deleting an entire table (relation) information [Student ID is known, but his/her password is unknown to an attacker]

IIIT Hyderabad

To obtain your most recent CGPA enter your studentID
and password below

Student ID:

Password:

The query built from the inputs supplied by the attacker is

```
select  S_ID, CGPA
from    students14
where   S_ID = 123; DROP table students17; - -
        and password = 'abc'
```

SQL Injection Remedies

- One mitigation strategy is to parse user input and reject input with symbols such as ' (single quote), " (double quote), ; (semi-colon), etc.

However, the input string containing `1 = 1` that appeared in one of the above queries may not get filtered out.

On the other hand, some names (such as O'Neal) have quotes in them and such user input may be wrongly rejected.

- Another approach is to specify what is acceptable rather than what is not.

This can be done through the use of regular expressions.

Reference:

<http://www.unixwiz.net/techtips/sql-injection.html>

Thank You !!!